



Lab Sheet 6

Filter Design using MATLAB

Prerequisite

Before starting this section, students should know..

1. A fundamental knowledge on MATLAB
2. A theoretical knowledge on Digital Filter.

Outcomes

After finishing this lab students should be able to.....

1. design digital filter using MATLAB.
2. understand different characteristics of Digital Filter.

Outlines of this Lab

1. Lab Session 6.1: Implementation of linear phase FIR filter
 - 6.1.1 Introduction
 - 6.1.2 Specification of a lowpass filter
 - 6.1.3 Types of linear phase FIR filter
 - 6.1.4 MATLAB Functions to implement Linear Phase FIR Filters
- 2 Lab Session 6.2: Window Design Techniques
 - 6.2.1 Introduction
 - 6.2.2 Commonly used windows
 - 6.2.3 MATLAB built in functions to design FIR filter using windows
 - 6.2.4 Design Example
- 3 Lab Session 6.3: In Lab Evaluation
- 4 Home work

Lab Session 6.1: Implementation of linear phase FIR filter

6.1.1. Introduction

In many applications like speech or audio signal processing, digital filters are used to implement frequency-selective operations. Therefore, specifications are required in the frequency-domain in terms of the desired magnitude and phase response of the filter. Generally, a linear phase response in the passband is desirable. In the case of FIR filters, it is possible to have exact linear phase. In the case of IIR filters a linear phase in the passband is not achievable. Hence we will consider magnitude-only specifications.

The magnitude specifications in general provide requirements in decibels (dB), given by

$$\text{dB scale} = -20 \log_{10} \frac{|H(e^{j\omega})|}{|H(e^{j\omega})|_{\max}} \geq 0$$

This approach is the most popular one in practice and is used for both FIR and IIR filters.

6.1.2. Specification of a lowpass filter:

A typical absolute and relative specification of a lowpass filter are shown in Figure 6.1a,b in which

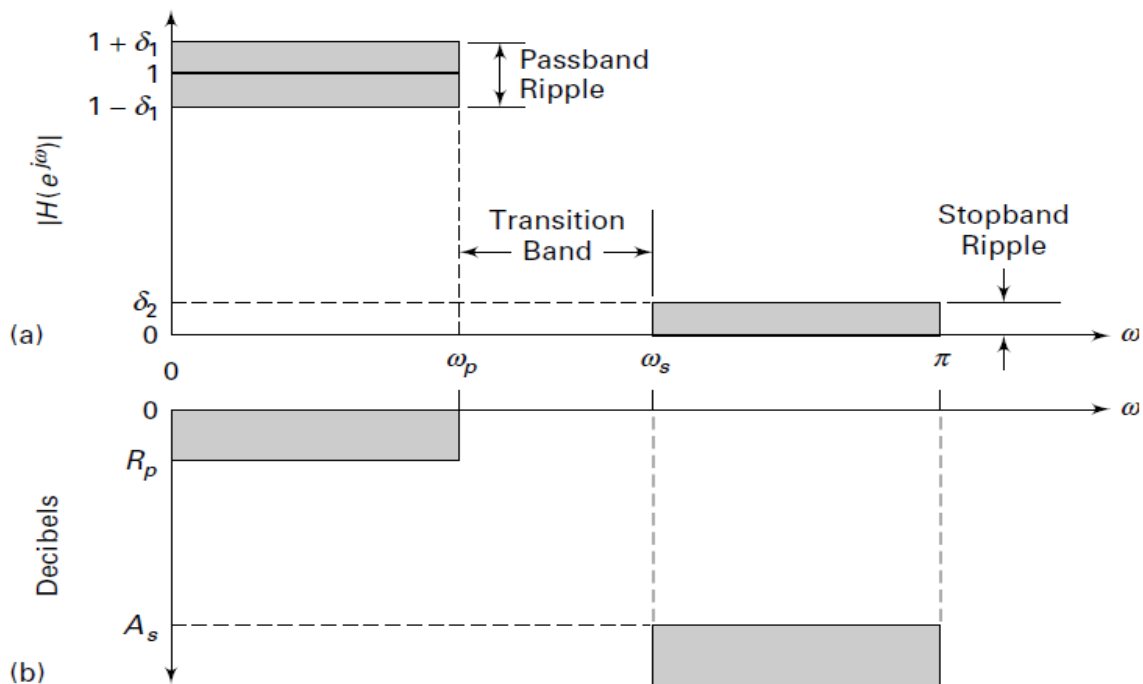


Fig 6.1: FIR filter specifications: (a) absolute (b) relative

A typical absolute specification of a lowpass filter is shown in Figure 6.1a, in which,

- (i) band $[0, \omega_p]$ is called the **passband**, and δ_1 is the **tolerance** (or ripple) that we are

- willing to accept in the ideal passband response,
- (ii) band $[\omega_s, \pi]$ is called the **stopband**, and δ_2 is the corresponding tolerance (or **ripple**),
 - (iii) band $[\omega_p, \omega_s]$ is called the **transition band**, and there are no restrictions on the magnitude response in this band.

A typical relative specification of a lowpass filter is shown in Figure 6.1b, in which

- (i) R_p is the passband ripple in dB.
- (ii) A_s is the stopband attenuation in dB.

The parameters given in these two specifications are obviously related.

Since $|H(e^{j\omega})|_{\max}$ in absolute specifications is equal to $(1 + \delta_1)$, we have

$$R_p = -20 \log_{10} \frac{1 - \delta_1}{1 + \delta_1} > 0 (\approx 0)$$

and

$$A_s = -20 \log_{10} \frac{\delta_2}{1 + \delta_1} > 0 (\gg 1)$$

Example 6.0 : In a certain filter's specifications the passband ripple is 0.25 dB, and the stopband attenuation is 50 dB. Determine δ_1 and δ_2 .

Solution:

$$R_p = 0.25 = -20 \log_{10} \frac{1 - \delta_1}{1 + \delta_1} \Rightarrow \delta_1 = 0.0144$$

$$A_s = 50 = -20 \log_{10} \frac{\delta_2}{1 + \delta_1} = -20 \log_{10} \frac{\delta_2}{1 + 0.0144} \Rightarrow \delta_2 = 0.0032$$

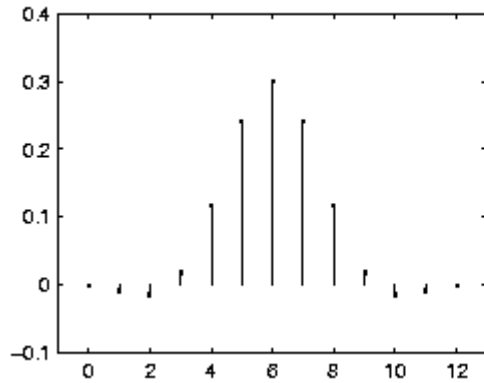
6.1.3 Types of linear phase FIR filter

FOUR TYPES OF LINEAR-PHASE FIR FILTERS

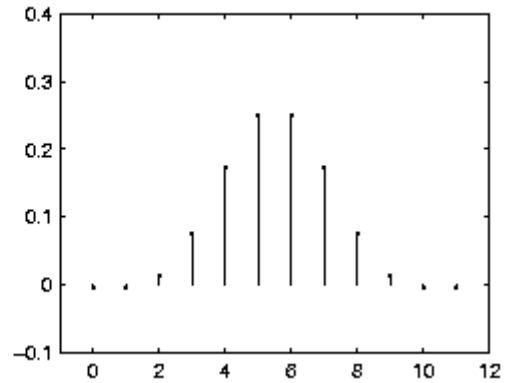
Linear-phase FIR filter can be divided into four basic types.

Type	impulse response	
I	symmetric	length is odd
II	symmetric	length is even
III	anti-symmetric	length is odd
IV	anti-symmetric	length is even

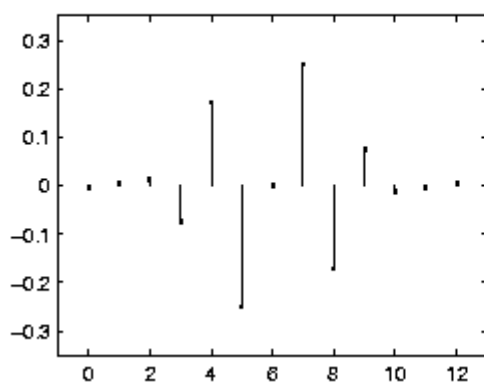
TYPE I IMPULSE RESPONSE



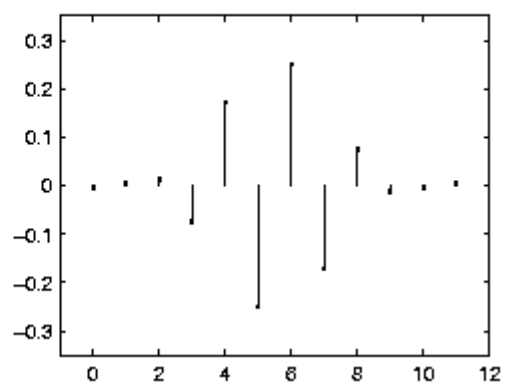
TYPE II IMPULSE RESPONSE



TYPE III IMPULSE RESPONSE



TYPE IV IMPULSE RESPONSE



Type-1 linear-phase FIR filter: Symmetrical impulse response, M odd In this case $\beta = 0$, $\alpha = (M - 1)/2$ is an integer, and $h(n) = h(M - 1 - n)$, $0 \leq n \leq M - 1$. Then we can show that

$$H(e^{j\omega}) = \left[\sum_{n=0}^{(M-1)/2} a(n) \cos \omega n \right] e^{-j\omega(M-1)/2}$$

where sequence $a(n)$ is obtained from $h(n)$ as

$$a(0) = h\left(\frac{M-1}{2}\right) \quad : \quad \text{the middle sample}$$

$$a(n) = 2h\left(\frac{M-1}{2} - n\right), \quad 1 \leq n \leq \frac{M-3}{2}$$

Comparing

$$H_r(\omega) = \sum_{n=0}^{(M-1)/2} a(n) \cos \omega n$$

Type-2 linear-phase FIR filter: Symmetrical impulse response, M even In this case again $\beta = 0$, $h(n) = h(M-1-n)$, $0 \leq n \leq M-1$, but $\alpha = (M-1)/2$ is not an integer. Then we can show that

$$H(e^{j\omega}) = \left[\sum_{n=1}^{M/2} b(n) \cos \left\{ \omega \left(n - \frac{1}{2} \right) \right\} \right] e^{-j\omega(M-1)/2}$$

where

$$b(n) = 2h \left(\frac{M}{2} - n \right), \quad n = 1, 2, \dots, \frac{M}{2}$$

Hence

$$H_r(\omega) = \sum_{n=1}^{M/2} b(n) \cos \left\{ \omega \left(n - \frac{1}{2} \right) \right\}$$

Note: At $\omega = \pi$ we get

$$H_r(\pi) = \sum_{n=1}^{M/2} b(n) \cos \left\{ \pi \left(n - \frac{1}{2} \right) \right\} = 0$$

regardless of $b(n)$ or $h(n)$. Hence we cannot use this type (i.e., symmetric $h(n)$, M even) for highpass or bandstop filters.

Type-3 linear-phase FIR filter: Antisymmetric impulse response, M odd In this case $\beta = \pi/2$, $\alpha = (M-1)/2$ is an integer, $h(n) = -h(M-1-n)$, $0 \leq n \leq M-1$, and $h((M-1)/2) = 0$. Then we can show

$$H(e^{j\omega}) = \left[\sum_{n=1}^{(M-1)/2} c(n) \sin \omega n \right] e^{j[\frac{\pi}{2} - (\frac{M-1}{2})\omega]}$$

where

$$c(n) = 2h \left(\frac{M-1}{2} - n \right), \quad n = 1, 2, \dots, \frac{M-1}{2}$$

and

$$H_r(\omega) = \sum_{n=1}^{(M-1)/2} c(n) \sin \omega n$$

Type-4 linear-phase FIR filter: Antisymmetric impulse response, M even This case is similar to Type-2. We have

$$H(e^{j\omega}) = \left[\sum_{n=1}^{M/2} d(n) \sin \left\{ \omega \left(n - \frac{1}{2} \right) \right\} \right] e^{j[\frac{\pi}{2} - \omega(M-1)/2]}$$

where

$$d(n) = 2h \left(\frac{M}{2} - n \right), \quad n = 1, 2, \dots, \frac{M}{2}$$

and

$$H_r(\omega) = \sum_{n=1}^{M/2} d(n) \sin \left\{ \omega \left(n - \frac{1}{2} \right) \right\}$$

Note: At $\omega = 0$, $H_r(0) = 0$ and $e^{j\pi/2} = j$. Hence this type is also suitable for designing digital Hilbert transformers and differentiators.

6.1.4. MATLAB Functions to implement Linear Phase FIR Filters

Type-1:

```
function [Hr,w,a,L] = Hr_Type1(h);
% Computes Amplitude response Hr(w) of a Type-1 LP FIR filter
% -----
% [Hr,w,a,L] = Hr_Type1(h)
% Hr = Amplitude Response
% w = 500 frequencies between [0 pi] over which Hr is computed
% a = Type-1 LP filter coefficients
% L = Order of Hr
% h = Type-1 LP filter impulse response
%
M = length(h); L = (M-1)/2;
a = [h(L+1) 2*h(L:-1:1)]; % 1x(L+1) row vector
n = [0:1:L]; % (L+1)x1 column vector
w = [0:1:500]'*pi/500; Hr = cos(w*n)*a';
end
```

Type-2

```
function [Hr,w,b,L] = Hr_Type2(h);
% Computes Amplitude response of a Type-2 LP FIR filter
% -----
% [Hr,w,b,L] = Hr_Type2(h)
% Hr = Amplitude Response
% w = frequencies between [0 pi] over which Hr is computed
% b = Type-2 LP filter coefficients
% L = Order of Hr
% h = Type-2 LP impulse response
%
M = length(h); L = M/2;
b = 2*[h(L:-1:1)]; n = [1:1:L]; n = n-0.5;
w = [0:1:500]'*pi/500; Hr = cos(w*n)*b';
end
```

Type-3

```
function [Hr,w,c,L] = Hr_Type3(h);  
% Computes Amplitude response Hr(w) of a Type-3 LP FIR filter  
% -----  
% [Hr,w,c,L] = Hr_Type3(h)  
% Hr = Amplitude Response  
% w = frequencies between [0 pi] over which Hr is computed  
% c = Type-3 LP filter coefficients  
% L = Order of Hr  
% h = Type-3 LP impulse response  
%  
M = length(h); L = (M-1)/2;  
c = [2*h(L+1:-1:1)]; n = [0:1:L];  
w = [0:1:500]'*pi/500; Hr = sin(w*n)*c';  
end
```

Type-4

```
function [Hr,w,d,L] = Hr_Type4(h);  
% Computes Amplitude response of a Type-4 LP FIR filter  
% -----  
% [Hr,w,d,L] = Hr_Type4(h)  
% Hr = Amplitude Response  
% w = frequencies between [0 pi] over which Hr is computed  
% d = Type-4 LP filter coefficients  
% L = Order of d  
% h = Type-4 LP impulse response  
%  
M = length(h); L = M/2;  
d = 2*[h(L:-1:1)]; n = [1:1:L]; n = n-0.5;  
w = [0:1:500]'*pi/500; Hr = sin(w*n)*d';  
end
```

Example 6.1:<Type-1 Linear Phase FIR Filter>

Let $h(n) = \{-4, 1, -1, -2, 5, 6, 5, -2, -1, 1, -4\}$. Determine the amplitude response $H_r(\omega)$ and the locations of the zeros of $H(z)$.

Solution:

Since $M = 11$, which is odd, and since $h(n)$ is symmetric about $\alpha = (11-1)/2 = 5$, this is a Type-1 linear-phase FIR filter. From (7.7) we have

$$a(0) = h(\alpha) = h(5) = 6, \quad a(1) = 2h(5-1) = 10, \quad a(2) = 2h(5-2) = -4$$

$$a(3) = 2h(5-3) = -2, \quad a(4) = 2h(5-4) = 2, \quad a(5) = 2h(5-5) = -8$$

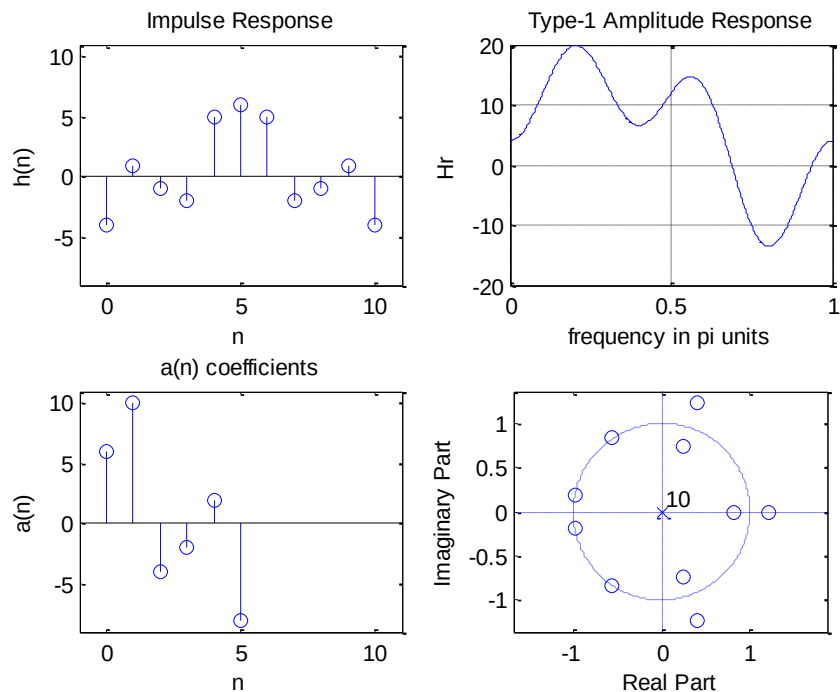
From (7.8), we obtain

$$\begin{aligned} H_r(\omega) &= a(0) + a(1) \cos \omega + a(2) \cos 2\omega + a(3) \cos 3\omega + a(4) \cos 4\omega + a(5) \cos 5\omega \\ &= 6 + 10 \cos \omega - 4 \cos 2\omega - 2 \cos 3\omega + 2 \cos 4\omega - 8 \cos 5\omega \end{aligned}$$

MATLAB Code:

```
h = [-4,1,-1,-2,5,6,5,-2,-1,1,-4];  
M = length(h); n = 0:M-1;  
[Hr,w,a,L] = Hr_Type1(h);  
amax = max(a)+1; amin = min(a)-1;  
subplot(2,2,1); stem(n,h); axis([-1 2*L+1 amin amax])  
xlabel('n'); ylabel('h(n)'); title('Impulse Response')  
subplot(2,2,2); plot(w/pi,Hr);grid  
xlabel('frequency in pi units'); ylabel('Hr')  
title('Type-1 Amplitude Response')  
subplot(2,2,3); stem(0:L,a); axis([-1 2*L+1 amin amax])  
xlabel('n'); ylabel('a(n)'); title('a(n) coefficients')  
subplot(2,2,4); zplane(h,1);
```

Output:



Similar problems:(Solve in lab)

Type-2:

Let $h(n) = \{-4, 1, -1, -2, 5, 6, 6, 5, -2, -1, 1, -4\}$. Determine the amplitude response $H_r(\omega)$ and the locations of the zeros of $H(z)$.

Type-3:

Let $h(n) = \{-4, 1, -1, -2, 5, 0, -5, 2, 1, -1, 4\}$. Determine the amplitude response $H_r(\omega)$ and the locations of the zeros of $H(z)$.

Type-4:

Let $h(n) = \{-4, 1, -1, -2, 5, 6, -6, -5, 2, 1, -1, 4\}$. Determine the amplitude response $H_r(\omega)$ and the locations of the zeros of $H(z)$.

6.1.5 FIR Filter Design using fir1 function

The MATLAB function `fir1()` designs conventional lowpass, highpass, bandpass, and bandstop linear-phase FIR filters based on the windowing method. The command:

`b = fir1(N,Wn)`

returns in vector **b** the impulse response of a **lowpass filter** of **order n**. The cut-off frequency **wn must be between 0 and 1** with 1 corresponding to the half sampling rate.

```
%% FIR design
a = 1;
b = fir1(20,0.2); %20th Low pass filter, 0.2 is cutoff freq
freqz(b,a,512)
%% example Application
load ecg
subplot(211);plot(ecg)
subplot(212);plot(filter(b,a,ecg))
% Try different orders
```

The command: **`b = fir1(N,Wn,'high')`** returns the impulse response of a **highpass filter** of order N with normalized cutoff frequency Wn.

Similarly, **`b = fir1(N,Wn,'stop')`** with Wn a two-element vector designating the stopband designs a bandstop filter.

Without explicit specification, the **Hamming window** is employed in the design. Other windowing functions can be used by specifying the windowing function as an extra argument of the function. For example, Blackman window can be used instead by the command **`b = fir1(N, Wn, blackman(N))`**.

```
%% HP FIR design
a = 1;
b = fir1(100,0.5,'high'); %100th High pass filter
freqz(b,a,512)
```

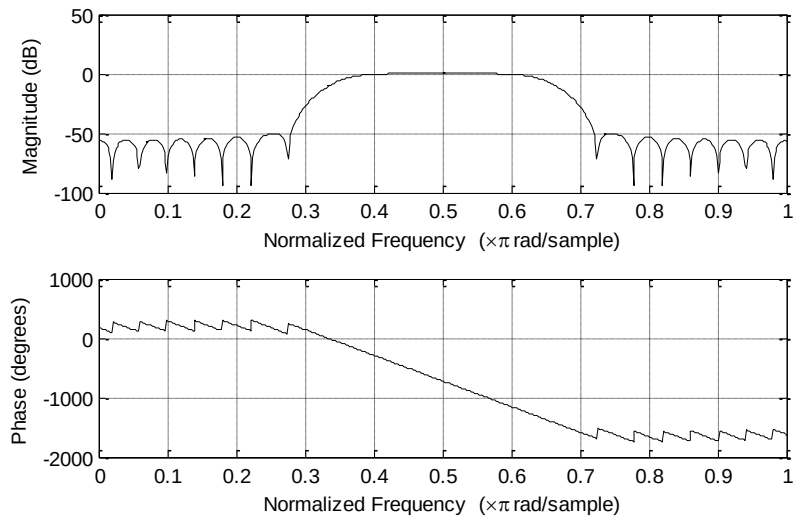
Example 6.2

Design a 48th-order FIR bandpass filter with passband $0.35 \leq w \leq 0.65$.

MATLAB Code:

```
b = fir1(48,[0.35 0.65]);
freqz(b,1,512)
```

Output:



```
%% Example Application
load mystery1
subplot(211);plot(mystery1)
a = 1; b = fir1(200,[0.12 0.28]);
subplot(212); plot(filter(b,a,mystery1))

%% Band Stop FIR design
a = 1;
b = fir1(100,[0.35 0.65],'stop'); %100th order Band Stop filter
freqz(b,a,512)
```

Example 6.3

Design a lowpass filter with passband defined from 0 to 1 kHz and stopband defined from 1500 Hz to 4 kHz. Specify a passband ripple of 5% and a stopband attenuation of 40 dB. Use kaiserord FIR order estimator (lowpass, highpass, bandpass, multiband).

$[N, W_n, BTA, FILTYPE] = \text{kaiserord}(F, A, DEV, F_s)$ is the **approximate order N** , normalized frequency band edges W_n , Kaiser window beta parameter BTA and filter type $FILTYPE$ to be used by the `FIR1` function:

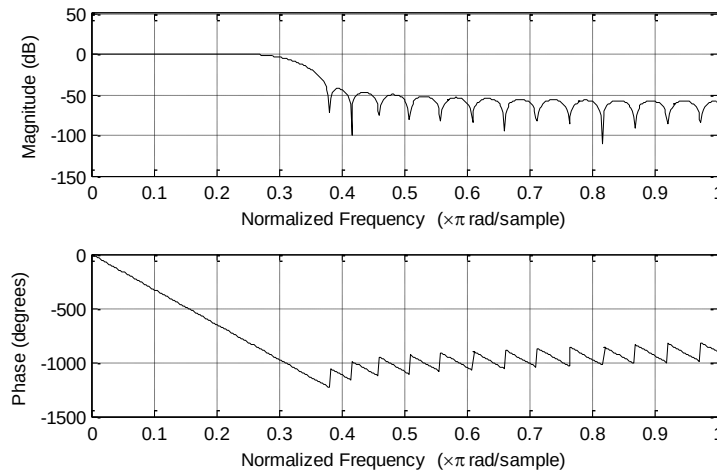
$B = \text{FIR1}(N, W_n, FILTYPE, \text{kaiser}(N+1, BTA), 'noscale')$

MATLAB Code:

```
%% Low pass filter using kaiserorder and kaiser window
fcuts = [1000 1500];
mags = [1 0];
devs = [0.05 0.01];
fsamp = 8000;
[n,Wn,beta,ftype] = kaiserord(fcuts,mags,devs,fsamp);
Win=kaiser(n+1,beta); % kaiser window
hh = fir1(n,Wn,ftype,Win,'noscale');
freqz(hh)
n
```

If empty or omitted, `firl` uses a Hamming window of length `n+1`.

Output:



`W = kaiser(N,BTA)` returns the BETA-valued **N-point Kaiser window**.

Task1: Try changing the parameters such as `fcuts` and `fsamp` and make comments in your report. There is a restriction between cut-off frequency and sampling frequency, can you find that ?

Task2: Use Rectangular, Hamming, Hanning and Blackman window instead of Kaiser window (use same window length). Which one is the best? and make comments in your report.

```
W=window(@rectwin,n+1)
W=window(@hamming,n+1)
W=window(@hann,n+1)
W=window(@blackman,n+1)
```

```
% Low pass filter using kaiserord and Rectangular window
fcuts = [1000 1500];
mags = [1 0];
devs = [0.05 0.01];
fsamp = 8000;
[n,Wn,beta,ftype] = kaiserord(fcuts,mags,devs,fsamp);
% using Rectangular Window
Win=window(@rectwin,n+1);
hh = firl(n,Wn,ftype,Win,'noscale');
freqz(hh)
```

Parks-McClellan FIR filter design

Parks-McClellan is the standard method for FIR filter design.

The MATLAB function `firpm()` designs a linear-phase FIR filter using the Parks-McClellan algorithm. The Parks-McClellan algorithm uses the Remez exchange algorithm and Chebyshev approximation theory to design filters with an optimal fit between the desired and

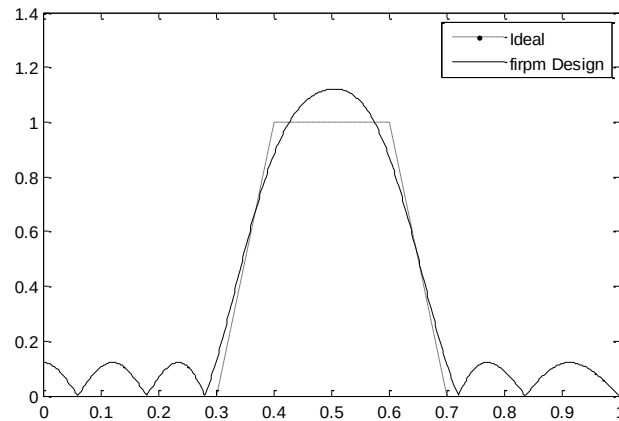
actual frequency responses. The filters are optimal in the sense that the maximum error between the desired frequency response and the actual frequency response is minimized.

Example 6.4: Graph the desired and actual frequency responses of a 17th-order Parks-McClellan bandpass filter:

MATLAB Code:

```
f = [0 0.3 0.4 0.6 0.7 1]; a = [0 0 1 1 0 0];  
b = firpm(17,f,a);  
[h,w] = freqz(b,1,512);  
plot(f,a,w/pi,abs(h))  
legend('Ideal','firpm Design')
```

Output:



Task1: Try changing the parameters and make comments in your report

Lab Session 6.2: Window Design Techniques

6.2.1 Introduction

Windows are used in the design of digital filters to **convert an "ideal" impulse response of infinite duration**, such as a sinc function, to a **finite impulse response (FIR)** filter design. In general, we need a IIR filter to get an ideal frequency response, but IIR is computationally expensive. So we move to FIR filter to save time, and it requires that we must truncate the impulse response at both ends with respect to the central. Even we truncate the impulse response when it is small enough, but such a sudden cutoff will cause some undesired effects. The window method will reduce them.

In the time domain, **windowing means that we multiply the desired (usually ideal) infinite duration impulse response $h_d(n)$ by a finite duration window (or window function) $w(n)$ to get a soft truncation.** The resulted impulse response $h(n)$ of the designed filter is the product

$$h(n) = h_d(n) w(n), \quad 0 \leq n \leq M$$

The basic idea behind the window design is to choose a proper ideal frequency-selective filter (which always has a noncausal, infinite-duration impulse response) and then to truncate (or window) its impulse response to obtain a linear-phase and causal FIR filter. Therefore the emphasis in this method is on selecting an appropriate *windowing* function and an appropriate *ideal* filter. We will denote an ideal frequency-selective filter by $H_d(e^{j\omega})$, which has a unity magnitude gain and linear-phase characteristics over its passband, and zero response over its stopband. An ideal LPF of bandwidth $\omega_c < \pi$ is given by

$$H_d(e^{j\omega}) = \begin{cases} 1 \cdot e^{-j\alpha\omega}, & |\omega| \leq \omega_c \\ 0, & \omega_c < |\omega| \leq \pi \end{cases}$$

where ω_c is also called the *cutoff* frequency, and α is called the sample *delay*. (Note that from the DTFT properties, $e^{-j\alpha\omega}$ implies shift in the positive n direction or delay.) The impulse response of this filter is of infinite duration and is given by

$$\begin{aligned} h_d(n) &= \mathcal{F}^{-1} [H_d(e^{j\omega})] = \frac{1}{2\pi} \int_{-\pi}^{\pi} H_d(e^{j\omega}) e^{j\omega n} d\omega \\ &= \frac{1}{2\pi} \int_{-\omega_c}^{\omega_c} 1 \cdot e^{-j\alpha\omega} e^{j\omega n} d\omega \\ &= \frac{\sin [\omega_c(n - \alpha)]}{\pi(n - \alpha)} \end{aligned}$$

Note that $h_d(n)$ is symmetric with respect to α , a fact useful for linear-phase FIR filters.

To obtain an FIR filter from $h_d(n)$, one has to truncate $h_d(n)$ on both sides. To obtain a causal and linear-phase FIR filter $h(n)$ of length M , we must have

$$h(n) = \begin{cases} h_d(n), & 0 \leq n \leq M-1 \\ 0, & \text{elsewhere} \end{cases} \quad \text{and} \quad \alpha = \frac{M-1}{2}$$

This operation is called “windowing.” In general, $h(n)$ can be thought of as being formed by the product of $h_d(n)$ and a window function $w(n)$ as follows:

$$h(n) = h_d(n)w(n)$$

where

$$w(n) = \begin{cases} \text{some symmetric function with respect to} \\ \alpha \text{ over } 0 \leq n \leq M-1 \\ 0, \text{ otherwise} \end{cases}$$

Depending on how we define $w(n)$, we obtain different window designs. For example, in (7.20)

$$w(n) = \begin{cases} 1, & 0 \leq n \leq M-1 \\ 0, & \text{otherwise} \end{cases} = \mathcal{R}_M(n)$$

which is the *rectangular window* defined earlier.

6.2.2 Commonly used windows

1 RECTANGULAR WINDOW

This is the simplest window function but provides the worst performance from the viewpoint of stopband attenuation. It was defined earlier by

$$w(n) = \begin{cases} 1, & 0 \leq n \leq M-1 \\ 0, & \text{otherwise} \end{cases}$$

Its frequency response function is

$$W(e^{j\omega}) = \left[\frac{\sin\left(\frac{\omega M}{2}\right)}{\sin\left(\frac{\omega}{2}\right)} \right] e^{-j\omega \frac{M-1}{2}} \Rightarrow W_r(\omega) = \frac{\sin\left(\frac{\omega M}{2}\right)}{\sin\left(\frac{\omega}{2}\right)}$$

2 BARTLETT WINDOW

Since the Gibbs phenomenon results from the fact that the rectangular window has a sudden transition from 0 to 1 (or 1 to 0), Bartlett suggested a more gradual transition in the form of a triangular window, which is given by

$$w(n) = \begin{cases} \frac{2n}{M-1}, & 0 \leq n \leq \frac{M-1}{2} \\ 2 - \frac{2n}{M-1}, & \frac{M-1}{2} \leq n \leq M-1 \\ 0, & \text{otherwise} \end{cases}$$

3 HANN WINDOW

This is a raised cosine window function given by

$$w(n) = \begin{cases} 0.5 \left[1 - \cos \left(\frac{2\pi n}{M-1} \right) \right], & 0 \leq n \leq M-1 \\ 0, & \text{otherwise} \end{cases}$$

4 HAMMING WINDOW

This window is similar to the Hann window except that it has a small amount of discontinuity and is given by

$$w(n) = \begin{cases} 0.54 - 0.46 \cos \left(\frac{2\pi n}{M-1} \right), & 0 \leq n \leq M-1 \\ 0, & \text{otherwise} \end{cases}$$

5 BLACKMAN WINDOW

This window is also similar to the previous two but contains a second harmonic term and is given by

$$w(n) = \begin{cases} 0.42 - 0.5 \cos \left(\frac{2\pi n}{M-1} \right) + 0.08 \cos \left(\frac{4\pi n}{M-1} \right), & 0 \leq n \leq M-1 \\ 0, & \text{otherwise} \end{cases}$$

6 KAISER WINDOW

This is an adjustable window function that is widely used in practice. The window function is due to J. F. Kaiser and is given by

$$w(n) = \frac{I_0 \left[\beta \sqrt{1 - \left(1 - \frac{2n}{M-1} \right)^2} \right]}{I_0 [\beta]}, \quad 0 \leq n \leq M-1$$

Formula for calculation of M and β

$$\text{transition width} = \Delta\omega = \omega_s - \omega_p$$

$$\text{Filter length } M \simeq \frac{A_s - 7.95}{2.285\Delta\omega} + 1$$

$$\text{Parameter } \beta = \begin{cases} 0.1102(A_s - 8.7), & A_s \geq 50 \\ 0.5842(A_s - 21)^{0.4} \\ \quad + 0.07886(A_s - 21), & 21 < A_s < 50 \end{cases}$$

6.2.3 MATLAB built in functions to design FIR filter using windows

MATLAB provides several functions to implement window functions discussed in this section. A brief description of these functions follow.

Function name	Return value
w=boxcar(M)	returns the M-point rectangular window function in array w.
w=bartlett(M)	returns the M-point Bartlett window function in array w.
w=hann(M)	returns the M-point Hann window function in array w.
w=hamming(M)	returns the M-point Hamming window function in array w.
w=blackman(M)	returns the M-point Blackman window function in array w.
w=kaiser(M,beta)	returns the beta-valued M-point rectangular window function in array

6.2.4 Design Example:

First we create the following two functions :

The ideal_lp function

```
function hd = ideal_lp(wc,M);
% Ideal LowPass filter computation
% -----
% [hd] = ideal_lp(wc,M)
% hd = ideal impulse response between 0 to M-1
% wc = cutoff frequency in radians
% M = length of the ideal filter
%
alpha = (M-1)/2; n = [0:1:(M-1)];
m = n - alpha; fc = wc/pi; hd = fc*sinc(fc*m);
end
```


The **freqz_m** function

```
function [db,mag,pha,grd,w] = freqz_m(b,a);
% Modified version of freqz subroutine
% -----
% [db,mag,pha,grd,w] = freqz_m(b,a);
% db = Relative magnitude in dB computed over 0
to pi radians
% mag = absolute magnitude computed over 0 to
pi radians
% pha = Phase response in radians over 0 to pi
radians
% grd = Group delay over 0 to pi radians
% w = 501 frequency samples between 0 to pi
radians
% b = numerator polynomial of H(z) (for FIR:
b=h)
% a = denominator polynomial of H(z) (for FIR:
a=[1])
%
[H,w] = freqz(b,a,1000,'whole');
H = (H(1:1:501))'; w = (w(1:1:501))';
mag = abs(H); db =
20*log10((mag+eps)/max(mag));
pha = angle(H); grd = grpdelay(b,a,w);
end
```

Example 6.5: Design a digital FIR lowpass filter with the following specifications:

$$\omega_p = 0.2\pi, \quad R_p = 0.25 \text{ dB}$$

$$\omega_s = 0.3\pi, \quad A_s = 50 \text{ dB}$$

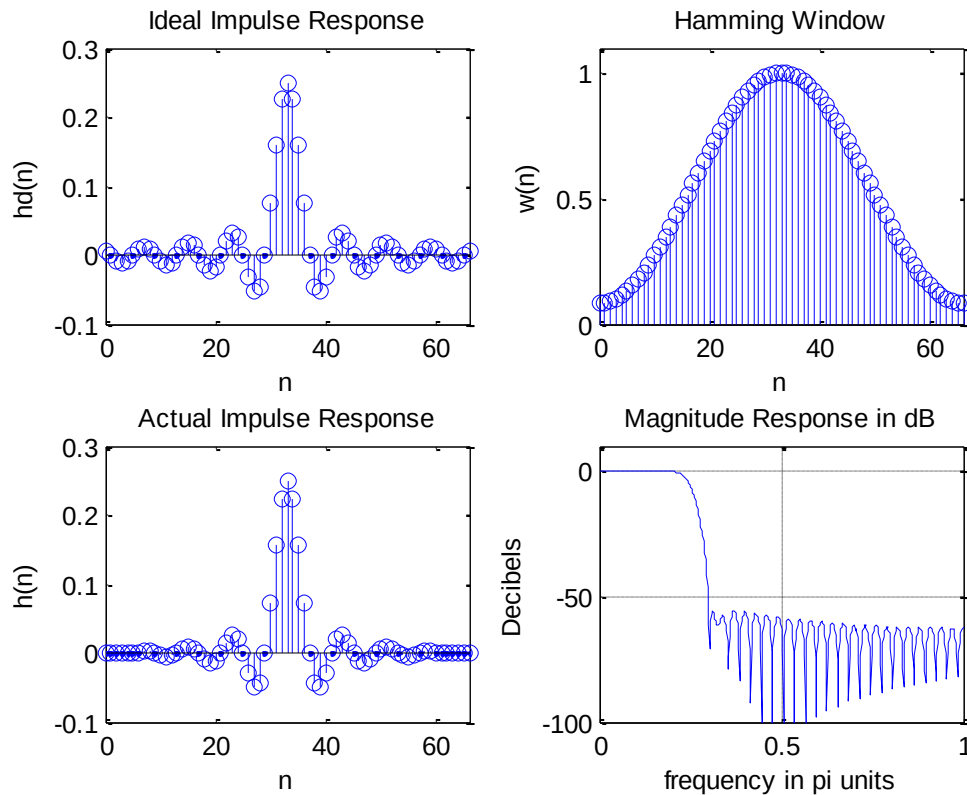
Choose Hamming window function. Determine the impulse response and provide a plot of the frequency response of the designed filter.

Solution:

MATLAB Code:

```
wp = 0.2*pi; ws = 0.3*pi; tr_width = ws - wp;
M = ceil(6.6*pi/tr_width) + 1;
n=[0:1:M-1];
wc = (ws+wp)/2, % Ideal LPF cutoff frequency
hd = ideal_lp(wc,M); w_ham = (hamming(M))'; h = hd .* w_ham;
[db,mag,pha,grd,w] = freqz_m(h,[1]); delta_w = 2*pi/1000;
Rp = -(min(db(1:1:wp/delta_w+1)));
As = -round(max(db(ws/delta_w+1:1:501)));
subplot(2,2,1); stem(n,hd); title('Ideal Impulse Response')
axis([0 M-1 -0.1 0.3]); xlabel('n'); ylabel('hd(n)')
subplot(2,2,2); stem(n,w_ham); title('Hamming Window')
axis([0 M-1 0 1.1]); xlabel('n'); ylabel('w(n)')
subplot(2,2,3); stem(n,h); title('Actual Impulse Response')
axis([0 M-1 -0.1 0.3]); xlabel('n'); ylabel('h(n)')
subplot(2,2,4); plot(w/pi,db); title('Magnitude Response in dB'); grid
axis([0 1 -100 10]); xlabel('frequency in pi units');
ylabel('Decibels')
```

Output:



At command window:

M = 67 $wc = 0.7854$ As = 52

Comment: Note that the filter length is $M = 67$, the actual stopband attenuation is 52 dB, and the actual passband ripple is 0.0394 dB. Clearly, the passband ripple is satisfied by this design. This practice of verifying the passband ripple is strongly recommended.

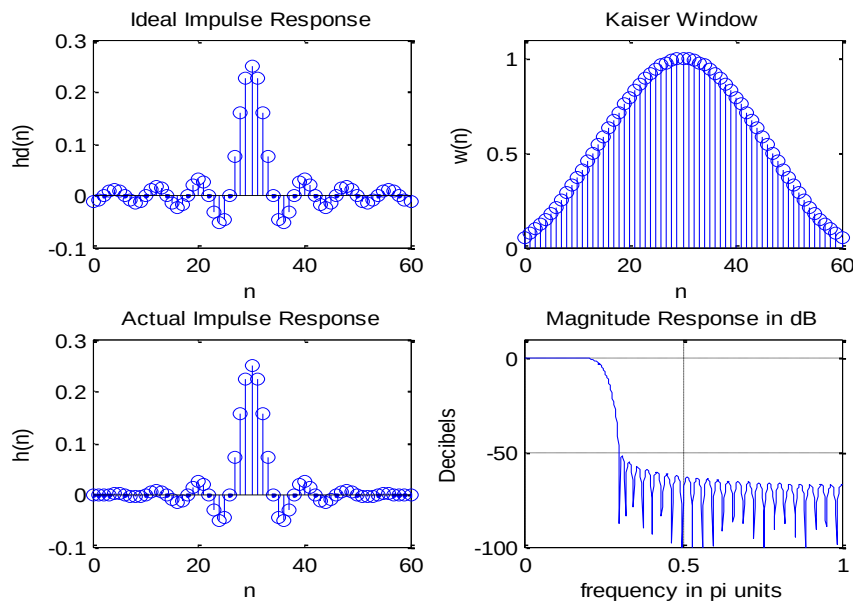
Example 6.6:

Repeat the Example using Kaiser window.

MATLAB Code:

```
wp = 0.2*pi; ws = 0.3*pi; As = 50; tr_width = ws - wp;
M = ceil((As-7.95)/(2.285*tr_width)+1) + 1
n=[0:1:M-1]; beta = 0.1102*(As-8.7)
wc = (ws+wp)/2; hd = ideal_lp(wc,M);
w_kai = (kaiser(M,beta))'; h = hd .* w_kai;
[db,mag,pha,grd,w] = freqz_m(h,[1]); delta_w = 2*pi/1000;
As = -round(max(db(ws/delta_w+1:1:501))) % Min Stopband Attenuation
subplot(2,2,1); stem(n,hd); title('Ideal Impulse Response')
axis([0 M-1 -0.1 0.3]); xlabel('n'); ylabel('hd(n)')
subplot(2,2,2); stem(n,w_kai);title('Kaiser Window')
axis([0 M-1 0 1.1]); xlabel('n'); ylabel('w(n)')
subplot(2,2,3); stem(n,h);title('Actual Impulse Response')
axis([0 M-1 -0.1 0.3]); xlabel('n'); ylabel('h(n)')
subplot(2,2,4);plot(w/pi,db);title('Magnitude Response in dB');grid
axis([0 1 -100 10]); xlabel('frequency in pi units');
ylabel('Decibels')
```

Output:



At Command window: $M = 61$ $\beta = 4.5513$ $A_s = 52$

Example 6.7: Design the following digital bandpass filter using Blackman window.

lower stopband edge: $\omega_{1s} = 0.2\pi$, $A_s = 60$ dB

lower passband edge: $\omega_{1p} = 0.35\pi$, $R_p = 1$ dB

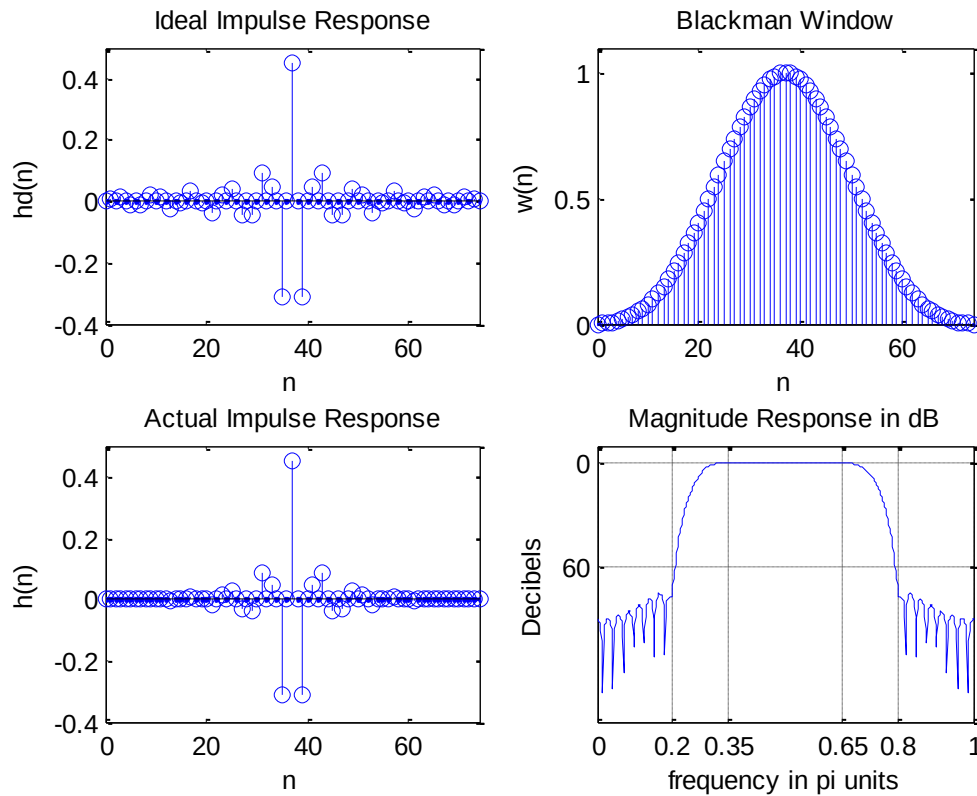
upper passband edge: $\omega_{2p} = 0.65\pi$, $R_p = 1$ dB

upper stopband edge: $\omega_{2s} = 0.8\pi$, $A_s = 60$ dB

Solution: MATLAB Code:

```
ws1 = 0.2*pi; wp1 = 0.35*pi; wp2 = 0.65*pi; ws2 = 0.8*pi; As = 60;
tr_width = min((wp1-ws1), (ws2-wp2))
M = ceil(11*pi/tr_width) + 1
n=[0:M-1]; wc1 = (ws1+wp1)/2; wc2 = (wp2+ws2)/2;
hd = ideal_lp(wc2,M) - ideal_lp(wc1,M);
w_bla = (blackman(M))';
h = hd .* w_bla; [db,mag,pha,grd,w] = freqz_m(h,[1]);delta_w = 2*pi/1000;
Rp = -min(db(wp1/delta_w+1:wp2/delta_w)) % Actual Passband Ripple
As = -round(max(db(ws2/delta_w+1:1:501))) % Min Stopband Attenuation
subplot(2,2,1); stem(n,hd); title('Ideal Impulse Response')
axis([0 M-1 -0.4 0.5]); xlabel('n'); ylabel('hd(n)')
subplot(2,2,2); stem(n,w_bla);title('Blackman Window')
axis([0 M-1 0 1.1]); xlabel('n'); ylabel('w(n)')
subplot(2,2,3); stem(n,h);title('Actual Impulse Response')
axis([0 M-1 -0.4 0.5]); xlabel('n'); ylabel('h(n)')
subplot(2,2,4);plot(w/pi,db);%set(gca,'FontName','cmr12');
title('Magnitude Response in dB');grid;
xlabel('frequency in pi units'); ylabel('Decibels')
axis([0 1 -150 10]);
set(gca,'XTickMode','manual','XTick',[0,0.2,0.35,0.65,0.8,1])
set(gca,'YTickMode','manual','YTick',[-60,0])
set(gca,'YTickLabelMode','manual','YTickLabels',['60';' 0'])
```

Output:



At command window:

$r_width = 0.4712$, $M = 75$, $R_p = 0.0030$, $A_s = 75$

Lab Session 6.3: In Lab Evaluation

ILE6.1

The frequency response of an ideal bandstop filter is given by

$$H_e(e^{j\omega}) = \begin{cases} 1, & 0 \leq |\omega| < \pi/3 \\ 0, & \pi/3 \leq |\omega| \leq 2\pi/3 \\ 1, & 2\pi/3 < |\omega| \leq \pi \end{cases}$$

Using a Kaiser window, design a bandstop filter of length 45 with stopband attenuation of 60 dB.

Homework:

1. The frequency response of an ideal digital differentiator is given by

$$H_d(e^{j\omega}) = \begin{cases} j\omega, & 0 < \omega \leq \pi \\ -j\omega, & -\pi < \omega < 0 \end{cases}$$

Using a Hamming window of length 21, design a digital FIR differentiator. Plot the time- and the frequency-domain responses.

2. Design a linear-phase bandpass filter using the Hann window design technique. The specifications are

$$\begin{aligned} \text{lower stopband edge: } & 0.2\pi & A_s &= 40 \text{ dB} \\ \text{upper stopband edge: } & 0.75\pi \\ \text{lower passband edge: } & 0.35\pi & R_p &= 0.25 \text{ dB} \\ \text{upper passband edge: } & 0.55\pi \end{aligned}$$

Plot the impulse response and the magnitude response (in dB) of the designed filter. Do not use the `fir1` function.

3. Design a highpass filter using one of the fixed window functions. The specifications are

$$\begin{aligned} \text{stopband edge: } & 0.4\pi, & A_s &= 50 \text{ dB} \\ \text{passband edge: } & 0.6\pi, & R_p &= 0.004 \text{ dB} \end{aligned}$$

Plot the zoomed magnitude response (in dB) of the designed filter in the passband to verify the passband ripple R_p . Do not use the `fir1` function.

4. Using the Kaiser window method, design a linear-phase FIR digital filter that meets the following specifications

$$\begin{aligned} 0.975 &\leq |H(e^{j\omega})| \leq 1.025, & 0 &\leq \omega \leq 0.25\pi \\ 0 &\leq |H(e^{j\omega})| \leq 0.005, & 0.35\pi &\leq \omega \leq 0.65\pi \\ 0.975 &\leq |H(e^{j\omega})| \leq 1.025, & 0.75\pi &\leq \omega \leq \pi \end{aligned}$$

Determine the minimum length impulse response $h(n)$ of such a filter. Provide a plot containing subplots of the amplitude response and the magnitude response in dB. Do not use the `fir1` function.